BINARY EXPLOITATION

Stack Layout (grows downward, high -> low addresses):
[caller frame]
[return address] <- overwrite to hijack control flow
[saved base pointer]
[stack canary] <- stack tampering detection
[local variables] <- overwrite to change program logic
[buffer buf[N[] <- overflow starts here

Buffer Overflow Goals:

1. Overwrite a local variable – fill buffer past its end to reach adjacent variable (e.g. flip is_admin =0 -> 1)
2. Overwrite the return address – keep writing past local vars and save base ptr until you reach return address of shellcode
3. Get code execution – Inject shellcode into the buffer, overwrite return addr to point to it -> when function returns, CPU executes shellcode

Finding Overflows: look for unchecked **strcpy**, **gets**, **memcpy**, **scanf("%s")**, or any loop writing into a fixed buffer without bounds checking

Shellcode Injection:

1. Craft shellcode bytes (machine code that spawns a shell) **2.** Write shellcode into buffer **3**. Overwrite return address with address of shellcode

4. Function returns -> CPU jumps to shellcode -> shell!

NOP Sled (0x90 = NOP instruction):

- Pad shellcode with a long run of NOP instructions before it
- If return address lands anywhere in the sled the execution slides forward into shellcode
- Compensates for ASLR uncertainty / imprecise address guessing
- Layout: [0x90 0x90 0x90 … 0x90 | shellcode | overwritten return address]

Defenses:

1. **Stack Canary** – Compiler inserts a secrett random value on the stack between buffer and return adr. Before function returns, it checks the canary is intact. IF overwritten -> crash/abort, BYPASSING requires leaking the canary value first.
2. **DEP / NX Bit (Data Execution Prevention / Non-Executable Stack):**
   - OS marks stack and heap pages as non-executable (Write XOR Execute = W⊕X)
   - If attacker jumps to shellcode on the stack -> CPU raises fault and kills processes
   - Prevents code injection attacks directly
   - Does not stop ROP (return oriented programming)
3. **ASLR** – Randomizes base addresses of stack, heap, and libraries each run. Makes hardcoded addresses unreliable. BYPASS: info leak, brute force (32-bit), or relative addressing
4. **Memory-Safe Languages** – Rust, Go enforce bounds checking at compile/runtime; eliminate entire class of buffer overflow bugs. US DARPA TRACTOR program aims to auto-convert C -> Rust.

Return Oriented Programming (ROP):

- Goal: bypass DEP/NX without injecting any new code
- Method: chain together existing code snippets ("gadgets") already in the binary or linked libraries
- Gadget = short sequence of instructions ending in ret (e.g. pop rdi; ret)
- Each gadget does one small operation; chaining them = arbitrary computation
- ROP chain sits on the stack; each gadget's address overwrites the "return address" for the previous one
- Tools: ROPgadget, pwntools
- Bypass stack canary + ASLR needed first (info leak usually required)

A ***vulnerability*** is some program weakness which can be exploited.

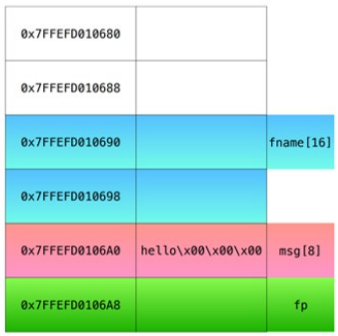
An ***exploit*** is an attack which exercises a vulnerability to achieve some goal.

Buffer Overflow

```c
void save_message(void) {
    char fname[16];
    char msg[8] = "hello";
    FILE *fp;

    printf("Enter filename: ");
    fgets(fname, FNAME_LEN, stdin);
    fname[strcspn(fname, "\n")] = '\0';

    fp = fopen(fname, "w");
    if (fp != NULL) {
        fputs(msg, fp);
        fclose(fp);
    }
}
```



NETWORKING

Protocol Stack Layers:

Layer	Protocol	Identifier
Application	HTTP, DNS, SSH	—
Transport	TCP / UDP	Port (0-65535)
Internet	IP / ARP / ICMP	IP Address
Link (Ethernet)	Ethernet / WIFI	MAC Address

Packets are nested: `[ Ethernet header | IP header | TCP header | App data ]`

TCP – Transmission Control Protocol:

- Connection oriented, reliable, ordered, error-checked
- SYN = synchronize, ACK = acknowledge
- 3-Way Handshake:
  1. Client -> Server: SYN(seq = x) **2.** Server -> Client: SYN + ACK (seq = y, ack = x+1) **3.** Client -> Server: ACK(ack = y+1)
     - Data exchange begins after step 3, connection is now established

SEQ/ACK Update Rules:

- SEQ = the byte number of the first byte in this segment
- ACK = the next byte number the receiver expects
- After sending N bytes: new SEQ = old SEQ + N
- Receiver replies with ACK = sender's SEQ + N (conforming receipt)
- Handshake: SEQ increments by 1 for SYN/FIN (treated as 1 byte)

UDP – User Datagram Protocol:

- Stateless, no handshake, no guaranteed delivery or ordering
- Lightweight and fast; used for DNS, video streaming, gaming
- No connection state = easily spoofed (attacker can forge source IP)

ARP – Address Resolution Protocol:

- Maps IP address -> MAC address on a local network (LAN only)
- ARP Request – broadcast to all: "Who has IP 10.0.0.2? Tell 10.0.0.1"
- ARP Reply – unicast "10.0.0.2 is at MAC f2:a5:7c:00:df:17"
- Gratuitous ARP (GARP) – unsolicited announcement of own IP -> MAC mapping; used to update caches
- Host maintain an ARP cache of IP -> MAC mappings to avoid repeated requests
- No authentication – any host can send any ARP message

Spoofing Attacks:

ARP:

1. Attacker sends fake GARP: "IP 10.0.0.3 -> attacker's MAC" **2.** Victim updates ARP cache with wrong mapping
3. Victim sends packets to attacker instead of real host **4.** Used as stepping stone for man in the middle.

UDP:

1. UDP has o connection state, so source IP/port is unverified **2.** Attacker forges source IP in UDP packet header
3. Recipient can't easily distinguish fake from real sender **4.** Used in DoS reflection/amplification attacks

IP:

1. Forge source IP address in IP packet header **2.** Responses go to spoofed IP(victim), not attacker
3. Enables DDoS amplification (send small query from victim's IP -> large response floods victim)

Man-in-the-middle (MITM):

1. ARP spoof both hosts: tell Host A "Host B's MAC = mine" AND tell Host B "Host A's MAC = mine"
2. Both hosts now route traffic through attackers machine **3.** Attacker reads and/or modifies packets in transit
4. Enable IP forwarding so packets still reach destination (stealth) **5.** Victims see no interruption, attacker sees all traffic

Network Scanning & Monitoring:

Active Scanning (generates traffic):

- **nmap** – port scanning, host discovery, OS detection, service version enumeration
- **ping** – ICMP echo: is host alive? Whats the RTT? • **netcat (nc)** – test TCP/UDP port connectivity; open raw connection:
- **scapy** – build and send custom packets at any layer (Python)

Passive Monitoring/Sniffing (no traffic generated):

- **Wireshark** – GUI packet capture and protocol dissection • **tcdump** – command-line packet capture
- Can see: source/destination IPs, protocols in use, TCP handshakes, unencrypted data (DNS queries, plain HTTP)
- Cannot easily see traffic on a switched network (switch sends frames only to correct port) – need MITM/ARP spoof first

DoS/DDoS:

- Goal: exhaust server resources (sockets, threads, memory, file descriptions)
- TCP SYN Flood – send many SYN packets, never complete handshake; server holds half-open connections until resources exhausted
- DDoS Reflection – spoof victim's IP, send queries to open DNS/NTP servers; amplified responses all hit victim

CRYPTOGRAPHY

Goals:

1. Confidentiality: keep data private **2.** Integrity: Ensure data wasn't modified in transit **3.** Authentication: verify the identity of sender

Key Terms:

1. Plaintext/Cleartext – unencrypted data **2.** Ciphertext – encrypted data **3.** Key – secret value used to encrypt and/or decrypt

Symmetric Encryption:

- Same key encrypts and decrypts • Fast; problem = how to securely share the key

One-Time PAD (OTP) – Theoretically unbreakable IF: key is at least as long as plaintext, truly random, never reused, kept secret

- Reuse attack: if key K is reused C1 ⊕ C2 = P1 ⊕ P2 -> attacker gets XOR of plaintexts, which leaks information

AES (Advanced Encryption Standard)

- NIST-approved symmetric block cypher • Block size: 128 bits (16 bytes) fixed • Key sizes: 128,192, or 256 bits
- Internally: SubBytes -> ShiftRows -> MixColumns -> AddRoundKey (repeated rounds)

XOR Operator (⊕): 0 ⊕ 0 = 0, 1 ⊕ 0 = 1 • Self-Inverse: (A ⊕ B) ⊕ B = A • Used in OTP, stream ciphers, and CBC chaining

Asymmetric (Public-Key) Encryption:

- Key pair: public key (share freely) + private key (keep secret)
- Encrypt with recipients public key -> only they can decrypt with private key
- Sign with senders private key -> anyone verifies with public key
- Solves key distribution issue of symmetric cypto
- Examples: RSA and ECC (Elliptic Curve Cryptography

Cryptographic Hash Functions:

- Maps arbitrary-length input -> fixed-size digest (e.g. SHA-256 -> 256 bits)
- One-way hash function; used for password storage, file integrity, digital signatures

3 Required Security Properties:

1. Collision Resistance – computationally infeasible to find any two inputs x != y where H(x) = H(y)
2. Preimage Resistance – given hash h, cannot find any x such that H(x) = h (one-way)
3. Second Preimage Resistance – given a specific x, cannot find any y != x such that H(x) = H(y)
4. Examples: SHA-256, SHA-3(secure); MD5, SHA-1 (broken do not use)

Digital Signatures (Integrity + Authentication):

- Sign: hash the message -> encrypt hash with senders private key -> attach as signature
- Verify: decrypt signature with senders public key -> recompute hash of message -> compare
- Provides Integrity, authentication, and non-repudiation

Block Ciphers:

Concept: split plaintext into fixed-size blocks, encrypt each with the key

PKCS#7 Padding:

- Block ciphers need full blocks (AES = 16 bytes) • Pad the last block by appending bytes whose value = # of padding bytes used
- 1 byte short -> append 0x01; 5 bytes short -> append 0x05 0x05 0x05 0x05 0x05
- If already full, add a complete extra block of 0x10 (16 times)

ECB – Electronic Codebook (Insecure):

Plaintext1 -> [Block Cipher + Key] -> Ciphertext1; Plaintext2 -> [Block Cipher + Key] -> Ciphertext2 (each block encrypted independently)

- No randomness, no chaining – same plaintext block always produces same ciphertext block
- Weakness: repeating patters in plaintext are visible in ciphertext • Encrypting a bitmap image with ECB still reveals the image outline
- Chosen Plaintext Attack (CPA) on ECB: attacker submits 15 known bytes + 1 unknown target byte -> receives ciphertext -> brute fores all 256 values of unknown byte -> finds match -> byte recovered. Repeat for entire message.

CBC – Cipher Block Chaining (better than ECB):

C_0 = IV (random initialization vector)

$C_i = E_k(P_i \oplus C_{i-1})$ <- Encryption

$P_i = D_k(C_i) \oplus C_{i-1}$ <- Decryption

IV ⊕ plaintext1 -> [encrypt+key] -> ciphertext1 ⊕ plaintext2 -> [encrypt+key] -> ciphertext2

- IV provides randomness – same plaintext -> different ciphertext each time
- Each block depends on all previous blocks • IV must be random and unpredictable (not secret, but not reused)

EBC vs. CBC:

Randomness: CBC; **Patterns Hidden**: CBC; **Parallel encrypt:** ECB; **Parallel Decrypt:** EBC & CBC; **IV needed:** CBC

CBC Tampering Attack:

- If attacker knows plaintext P and controls the IV(or previous ciphertext block):
- Compute intermediate value: I = IV ⊕ P (I is raw output of decryption before XOR)
- To produce desired plaintext P': set IV' = I ⊕ P' • Victim decrypts with tampered IV -> gets attackers chosen plaintext
- Works because $P_i = D_k(C_i) \oplus C_{i-1}$ – attacker controls the XOR input

Padding Oracle Attack:

Setup: Attacker has a ciphertext + an oracle(system that says "valid padding" or "invalid padding" after decryption attempt - even just error message or timing difference

Goal: Recover plaintext without the key; How it works byte by byte:

1. Attacker tweaks the last byte of the IV and sends to oracle **2.** Try all 256 values of that byte until oracle says "valid padding" -> means decrypted last byte = 0x01 (valid 1-byte PKCS#7 padding) **3.** Now you know D_k(Ciphertext)[last byte] = IV_guess ⊕ 0x01
4. This gives you the "intermediate" (I) value for that byte **5.** Recover plaintext byte: P = I ⊕ original_IV[last byte]
6. Repeat for second-to-last byte (now target padding 0x02 0x02), and so on **7.** Repeat across all blocks -> entire plaintext recovered

Why it works: decryption XORs intermediate value with the previous block (or IV); attacker controls that XOR input; valid-padding feedback leaks the intermediate value

Fix: use authenticated encryption (AES-GCM) – Never leak padding error details; always return a generic error with constant timing.

$$P_i = D_K(C_i) \oplus C_{i-1}$$

$$P_i = I_i \oplus C_{i-1}$$

PUBLIC-KEY CRYPTOGRAPHY

Why Asymmetric?

Symmetric crypto can't securely exchange keys over insecure channel. Solution: each party gets a **public key** (share freely) + **private key** (keep secret).

› Encrypt with recipient's **public key** → only they can decrypt with private key

› Sign with sender's **private key** → anyone verifies with public key

RSA — Key Generation Steps

```
1. Choose two large primes p, q
2. n = p × q              (modulus, public)
3. φ(n) = (p-1)(q-1)  (Euler's totient)
4. Choose e: 1<e<φ(n), gcd(e,φ(n))=1  (public exp)
5. d = (1 + k·φ(n)) / e  (private exp)

Public Key:  (n, e)
Private Key: (n, d)   ← keep p,q,φ(n),d secret
```

Example: p=53, q=59 → n=3127, φ(n)=3016, e=3, d=2011

RSA Encrypt / Decrypt

```
Encrypt: c = m^e mod n
Decrypt: m = c^d mod n

Example: m=89, e=3, n=3127
  c = 89³ mod 3127 = 1394
  m = 1394²⁰¹¹ mod 3127 = 89 ✓
```

Why RSA is Hard to Break

Eve sees (n, e, c). To decrypt she needs d, which requires φ(n), which requires factoring n into p and q. **Integer factorization is in NP** — no efficient algorithm exists for large n.

KEY SIZE Use 2048-bit keys (256 bytes). Longer key = exponentially harder to crack.

Diffie-Hellman Key Exchange (DHKE)

Establish shared secret over public channel without transmitting it. Analogy: mixing paint colors — easy to mix, hard to separate.

```
Public params: prime p=23, base g=5
Alice secret a=4: A = g^a mod p = 5⁴ mod 23 = 4
Bob secret b=3:   B = g^b mod p = 5³ mod 23 = 10
Alice: s = B^a mod p = 10⁴ mod 23 = 18
Bob:   s = A^b mod p = 4³ mod 23 = 18  ← same!
```

Limitation: RSA/DHKE can only encrypt data ≤ key length. Solution: use RSA to exchange a symmetric key (AES), then encrypt bulk data with AES.

Digital Signatures

Provides INTEGRITY AUTH NON-REPUDIATION

```
SIGN   (Bob):   sig = Encrypt(H(msg), Kbob_private)
VERIFY (Alice): H' = Decrypt(sig, Kbob_public)
                recompute H(msg); if H'==H → valid
```

› Only Bob's private key produces a sig that decrypts with his public key

› If message tampered → H(msg) changes → sig mismatch

Digital Certificates & PKI

Problem: how do you know a public key truly belongs to Bob? Anyone can claim to be Bob.

Certificate = authoritative document linking entity → public key, signed by a **Certification Authority (CA)** (DigiCert, Let's Encrypt, Comodo).

Browser checks site's cert against trusted CAs. "Connection not private" = cert invalid/unrecognized CA.

HTTPS / TLS Flow

```
1. Server sends certificate (public key)
2. Client verifies cert via CA chain
3. Client generates secret S
4. Encrypt S with server's public key → send
5. Server decrypts S with private key
6. Both derive session key K from S
7. Bulk data encrypted with K (AES)
```

Everything Together

SYMMETRIC (AES) bulk encryption

ASYMMETRIC (RSA) key exchange + signatures

HASHING (SHA-256) integrity + password storage

OpenSSL Quick Reference

```
openssl dgst -sha256 file.txt       # hash file
openssl genrsa -aes128 -out priv.pem 2048
openssl rsa -in priv.pem -pubout > pub.pem
openssl rsautl -encrypt -inkey pub.pem -pubin
openssl rsautl -decrypt -inkey priv.pem
```

HASHING

Cryptographic Hash Functions

Arbitrary-size input → fixed-size digest. One-way. SHA-256 → 256 bits.

3 Security Properties

Property	Meaning
Preimage Resistance	Given h, can't find x s.t. H(x)=h ("one-way")
2nd Preimage Resist.	Given x, can't find y≠x s.t. H(x)=H(y)
Collision Resistance	Can't find any x,y s.t. H(x)=H(y)

Hash Families

› **MD (Rivest)**: 128-bit. MD2/MD4 = broken. MD5 = collision broken, one-way OK (don't use). MD6 = not widely used.

› **SHA-0**: withdrawn. **SHA-1**: collision found 2017 (shattered.it). **SHA-2** (SHA-256/512): no significant attack. **SHA-3**: sponge function, not NSA-designed — genuine alternative.

Avalanche effect: 1 bit change → completely different hash. Internal structure: Merkle-Damgård (MD5/SHA-1/SHA-2); Sponge (SHA-3).

Applications

› INTEGRITY Hash file → send with message → receiver rehashes → different hash = tampered

› PASSWORDS Store H(password), never plaintext. Compare H(input) to stored hash at login.

› SALTING Append random salt before hashing → same password → different hash → defeats rainbow tables

› MAC H(message ‖ sharedKey) → proves integrity + authenticity to anyone with shared key

› COMMITMENT Share hash without revealing value (e.g. sealed bids). Reveal later → verifiable.

Attacks on Hashes

› BRUTE FORCE Hash all candidates; slow, not guaranteed

› DICT ATTACK Pre-hashed wordlist → compare → works on weak passwords

› RAINBOW TABLE Pre-computed plaintext-hash chains (Word→Hash→Reduce→...). Lookup fast; files huge. Defeated by salting.

RainbowCrack generates tables. Commercial tables sold for $550+. Brute=years; rainbow=weeks.